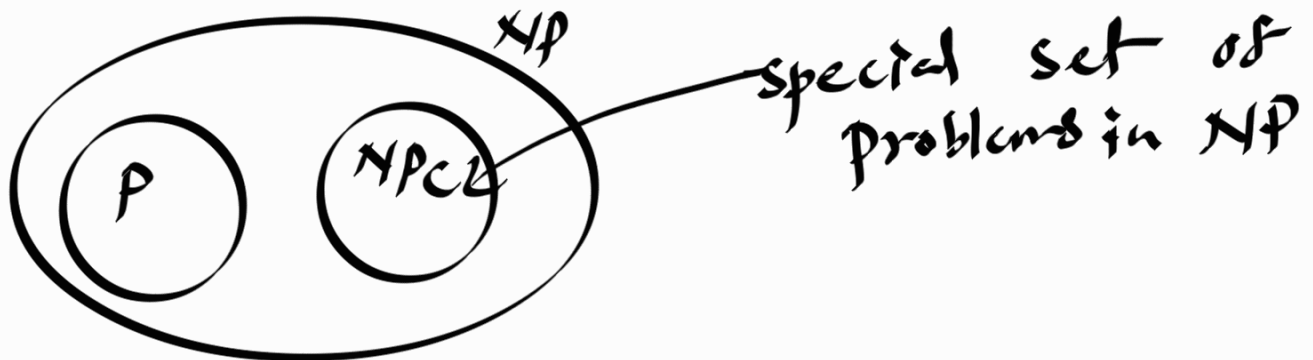


In 1970, Stephen Cook and Leonid Levin discovered that certain problems in NP whose individual complexities is related to that of the entire class.

If a poly-time solution exists for any of these problems, all problems in NP is polynomially solvable. These set of problems are called NP-complete (NPC) problems.



Why is this important?

- If you are trying to prove $P=NP$, then find a poly-time algo for one of the NPC problems.
- Proving a problem is NPC is a strong indication that the problem does not have a poly-time algorithm.

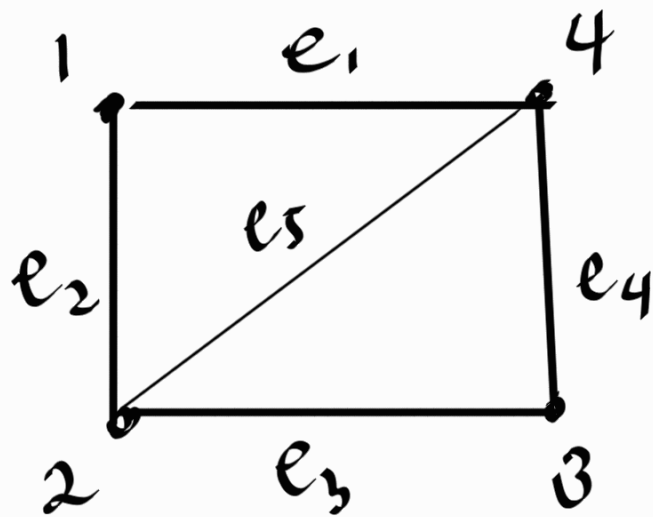
When we talk about Problems in NP, we only talk about decision problems.

- The decision version of a problem is equal to the corresponding optimization problem in terms of poly times solvability.

Example: Min-vertex cover

Given a graph $G = (V, E)$, find the smallest vertex cover

$VC \quad V' \subseteq V : \forall (u, v) \in E : (u \in V') \vee (v \in V')$



G

$\{2, 4\}$

$\{1, 2, 3\}$

I can rephrase the question.

Given $G = (V, E)$ and $k \in \mathbb{N}$, is there a VC of size at most k ?

$\langle G, 1 \rangle$ no

$\langle G, n \rangle$ yes

$\langle G, 2 \rangle$ yes

$\langle G, n-1 \rangle$ yes

$\langle G, 3 \rangle$ yes

$\langle G, n-2 \rangle$ yes

$\vdots$

$\langle G, k^* \rangle$ yes

$\langle G, k^*-1 \rangle$ no

Polynomial reductions

- These are map reductions that takes polynomial time.

Definition 7.28

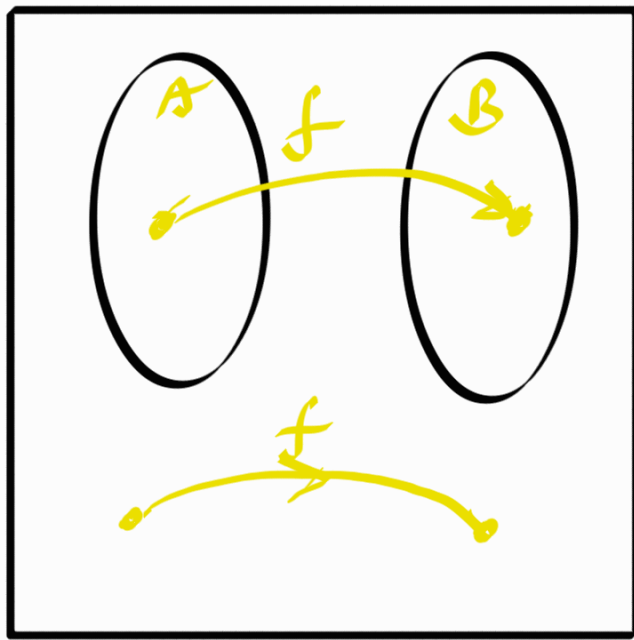
A function $f: \Sigma^* \rightarrow \Sigma^*$ is a polynomial time computable function if some polynomial time TM M exists that halts with just $f(w)$ on its tape, when started on any input w .

Definition 7.29

A language A is polynomial time mapping reducible, or polynomial time reducible, to language B , written $A \leq_p B$, if polynomial time computable function $f: \Sigma^* \rightarrow \Sigma^*$ exists, where every w ,

$$w \in A \iff f(w) \in B$$

The function f is called the polynomial time reduction of A to B .



Theorem 7.21,

If $A \leq_p B$ and $B \in P$, then $A \in P$

we will do polynomial reduction
examples later.

A language B is NP-complete if it satisfies the following two conditions.

1. $B \in NP$
2. Every problem in NP is polynomial time reducible to B .

Theorem 7.35

If B is NP-complete and $B \in P$, then $P = NP$

Theorem 7.36

If B is NP-complete and $B \leq_p C$ and $C \in NP$, then C is NP-complete.

$$A \leq_p B \leq_p C \quad C \in NP$$

The Cook-Levin theorem

Theorem 7.37

SAT is NP-complete

$$\text{SAT} = \{ \langle \phi \rangle \mid \phi \text{ is satisfiable boolean formula} \}$$

$$\phi = (x_1 \vee x_2 \vee \bar{x}_3 \vee x_4) \wedge (x_1 \vee \bar{x}_2 \vee x_3 \vee x_6 \vee x_7) \wedge \dots$$

$$x_1, x_2, x_3, x_4, x_5, x_6, x_7$$

$$\text{ex: } \psi = (x_1 \vee x_2 \vee x_3) \wedge (x_5 \vee \bar{x}_6)$$

$$x_1 = \top, x_5 = \top$$

$$\phi = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_2) \wedge (x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$$

This proof is very complicated.

First, they prove that SAT is in NP.

Then, using a clever argument, they show that any problem in NP can be reduced to a SAT instance in polynomial time.

If we have time we will go through the overview of this proof after Test 3.

You are given that SAT is NP-complete

If you are given a new problem B , and if you show that $SAT \leq_p B$ and $B \in NP$, then B is NP-complete

SAT is NP-complete

3-SAT is NP-complete.

3-SAT is a special case of satisfiability problem whereby all formulas are in special form.

A literal is a boolean variable or negated boolean variable, as in x or $\bar{x}$.

A clause is several literals connected with $\vee$, as in $(x_1 \vee \bar{x}_2 \vee \bar{x}_3 \vee x_4)$.

A Boolean formula is in conjunctive normal form, called a **cnf-formula**, if it comprises several clauses connected with $\wedge$, as in

$$(x_1 \vee \bar{x}_2 \vee \bar{x}_3 \vee x_4) \wedge (x_3 \vee \bar{x}_5 \vee x_6) \wedge (x_3 \vee \bar{x}_6)$$

It is a 3-cnf formula if all clauses has three literals, as in

$$(x_1 \vee \bar{x}_2 \vee \bar{x}_3) \wedge (x_4 \vee x_2 \vee x_5) \wedge (x_1 \vee \bar{x}_6 \vee x_4)$$

$$3\text{-SAT} = \{ \langle \phi \rangle \mid \phi \text{ is a satisfiable 3cnf formula} \}$$

Now we focus on polynomial time reductions.

Reason?

we are going to use these polynomial time reductions to show some problems are NP-complete. we will learn more theory about this later.